

**BABEȘ-BOLYAI UNIVERSITY, CLUJ-NAPOCA,
ROMANIA**

FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

Face2Learn

– MIRPR Report 2025 –

Team members:

Bucur Victor Sever, Popoviciu Luca, Porcar Cezar, Potra-Rațiu Darius, Preduca Matei

2025

Rezumat

Aplicația reprezintă un sistem inteligent bazat pe inteligență artificială, denumit **Face2Learn**, care combină modele de limbaj mari (LLM), sinteză vocală (TTS) și animație facială pentru a crea o experiență de învățare naturală și interactivă. Scopul proiectului este de a transforma informațiile academice în explicații conversaționale, vizuale și auditive, crescând astfel implicarea, accesibilitatea și retenția informației.

În versiunea finală, sistemul a fost migrat către o arhitectură Full-Stack modernă. Backend-ul (REST API cu FastAPI) este optimizat pentru latență scăzută și rulare multiplatformă (CUDA/Metal), utilizând mecanisme avansate de RAG (Hybrid Search, Cosine Similarity). Frontend-ul, dezvoltat în React și Vite, oferă o interfață grafică intuitivă, cu elemente de design modern (Glassmorphism) și feedback vizual în timp real.

Cuprins

1	Descrierea problemei rezolvate cu ajutorul AI	3
1.1	Contextul problemei	3
1.2	Scopul și importanța problemei	3
1.3	Utilizatorii sistemului	3
1.4	Datele de intrare și ieșire	4
1.5	Tipurile de date utilizate	4
1.6	Măsurarea performanței sistemului AI	4
2	Related work and useful tools and technologies	5
2.1	1. LoRA (Low-Rank Adaptation)	5
2.2	2. QLoRA	5
2.3	3. llama.cpp	5
2.4	4. TinyLLM (Phi, Mistral, TinyLLaMA)	6
2.5	5. RAGFlow	6
2.6	6. RAG-Anything	6
2.7	7. Whisper TTS	6
2.8	8. Piper	6
2.9	9. Wav2Lip	6
2.10	10. SadTalker	7
2.11	Resurse și Link-uri Oficiale ale Modelelor	7
3	Evaluarea sistemului RAG utilizat în Face2Learn	8
3.1	Descrierea și explorarea datelor (EDA)	8
3.2	Descrierea algoritmului inteligent	8
3.3	Metodologia experimentală	9
3.4	Rezultate obținute (Etapa Inițială - Baseline)	9
4	Îmbunătățirea și Optimizarea Algoritmului Inteligent	10
4.1	Optimizarea Sistemului RAG prin Two-Stage Retrieval	10
4.1.1	Etapa 1: Candidate Generation (Bi-Encoder)	10
4.1.2	Etapa 2: Reranking (Cross-Encoder)	10
4.2	Arhitectura de Căutare Hibridă (Ensembling)	11

4.3	Optimizarea Modelului Generativ (Integrare QLoRA)	11
4.4	Augmentarea Datelor și a Interogărilor	11
4.4.1	Query Expansion	11
4.5	Studiu Experimental al Hiperparametrilor	11
4.5.1	Schimbarea Modelului de Embedding	11
4.5.2	Analiza Temperaturii (T)	12
4.5.3	Analiza Chunk Size și Top-K	12
5	Arhitectura Sistemului și Detalii de Implementare Software	13
5.1	Arhitectura REST API cu FastAPI	13
5.1.1	Definirea Endpoint-urilor	13
5.2	Dezvoltarea Interfeței Grafice (Frontend)	13
5.2.1	Tehnologii Utilizate	14
5.2.2	Design și User Experience (UX)	14
5.2.3	Funcționalități Cheie Implementate	14
5.3	Optimizări de Infrastructură și DevOps	14
5.3.1	Unificarea Mediului de Dezvoltare (Single Venv)	14
5.3.2	Compatibilitate Multi-Platformă (CUDA & Metal)	15
5.3.3	Gestionarea Fișierelor Multimedia	15
5.4	Refactorizarea Codului (Code Cleaning)	15

Capitolul 1

Descrierea problemei rezolvate cu ajutorul AI

1.1 Contextul problemei

Sistemele educaționale tradiționale se bazează predominant pe text și prelegeri statice. În contextul actual al digitalizării, apare nevoia de metode de predare interactive, personalizate și accesibile. Proiectul **Face2Learn** propune utilizarea inteligenței artificiale multimodale pentru a crea un asistent virtual care explică concepte academice prin vorbire și expresii faciale sincronizate.

Modelul AI poate înțelege întrebări formulate în limbaj natural, poate accesa informații relevante din materiale educaționale și poate furniza răspunsuri clare, exprimate vocal și vizual printr-un avatar animat.

1.2 Scopul și importanța problemei

Scopul principal este de a transforma procesul de învățare într-o experiență naturală, captivantă și accesibilă. Importanța proiectului derivă din:

- creșterea **engagement-ului** și motivației elevilor/studentilor;
- asigurarea **accesibilității** pentru persoane cu deficiențe de vedere sau auz;
- sprijinirea cadrelor didactice prin automatizarea explicațiilor și a sesiunilor de Q&A;
- posibilitatea **învățării personalizate** în ritmul fiecărui utilizator.

1.3 Utilizatorii sistemului

- **Studenți și elevi** – folosesc avatarul AI pentru explicații și recapitulări;
- **Profesori și tutori** – utilizează sistemul pentru demonstrații și asistență automată;

- **Instituții educaționale** – integrează soluția pentru suport didactic 24/7;
- **Persoane cu dizabilități** – beneficiază de conținut multimodal adaptat.

1.4 Datele de intrare și ieșire

Date de intrare:

- întrebări formulate în limbaj natural (text sau voce);
- documente educaționale (manuale, cursuri, notițe);
- preferințe ale utilizatorului (limbă, voce, tonalitate).

Date de ieșire:

- răspuns text generat de LLM;
- voce sintetică naturală generată prin TTS;
- videoclip animat cu avatar sincronizat cu vorbirea.

1.5 Tipurile de date utilizate

- corpusuri textuale academice și explicații didactice;
- înregistrări audio pentru antrenarea TTS;
- imagini/video cu expresii faciale pentru sincronizare (lip-sync);
- embeddings semantice pentru căutare contextuală (RAG).

1.6 Măsurarea performanței sistemului AI

Performanța sistemului este evaluată prin indicatori cantitativi și calitativi:

- **Acuratețea răspunsurilor** – procentul de răspunsuri corecte;
- **Timpul mediu de răspuns** – durata procesării end-to-end;
- **Calitatea animației** – gradul de sincronizare buze-vorbire;
- **Consum de resurse** – memorie și timp de inferență.

Capitolul 2

Related work and useful tools and technologies

Această secțiune prezintă cele zece proiecte și tehnologii relevante analizate pentru construcția avatarului educațional. Pentru fiecare sunt menționate tipul datelor folosite, algoritmii utilizați și performanțele.

2.1 1. LoRA (Low-Rank Adaptation)

Date: text educațional.

Algoritmi: fine-tuning eficient al LLM-urilor prin adaptare low-rank.

Performanță: îmbunătățire a acurateței cu cost redus.

Tehnologii: PyTorch, Hugging Face.

2.2 2. QLoRA

Date: corpus text.

Algoritmi: fine-tuning cu cuantizare pentru reducerea memoriei.

Performanță: menține calitatea modelului la 4-bit.

Tehnologii: Transformers, bitsandbytes.

2.3 3. llama.cpp

Date: text.

Algoritmi: inferență locală pentru modele cuantizate (GGUF).

Performanță: latență redusă pe CPU/GPU consumer.

Tehnologii: C++, GGUF models.

2.4 4. TinyLLM (Phi, Mistral, TinyLLaMA)

Date: text.

Algoritmi: modele compacte pentru rulare eficientă.

Performanță: raport bun între viteză și acuratețe.

Tehnologii: PyTorch, Hugging Face.

2.5 5. RAGFlow

Date: documente și note de curs.

Algoritmi: RAG (retrieval augmented generation).

Performanță: răspunsuri mai relevante.

Tehnologii: LangChain, FAISS.

2.6 6. RAG-Anything

Date: fișiere locale (PDF, text).

Algoritmi: flux simplificat RAG.

Performanță: acces rapid la surse externe.

Tehnologii: Python, Streamlit.

2.7 7. Whisper TTS

Date: corpusuri audio și transcripții text.

Algoritmi: model neural de sinteză vocală bazat pe arhitectura Whisper.

Performanță: voce naturală și suport multilingv.

Tehnologii: PyTorch, Whisper TTS API (OpenAI).

2.8 8. Piper

Date: audio/text.

Algoritmi: TTS optimizat pentru dispozitive edge.

Performanță: latență foarte mică.

Tehnologii: Rust, on-device inference.

2.9 9. Wav2Lip

Date: video + audio.

Algoritmi: lip-sync bazat pe rețele CNN.

Performanță: aliniere buze-vorbire realistă.

Tehnologii: PyTorch, OpenCV.

2.10 10. SadTalker

Date: imagine + audio.

Algoritmi: talking-face generation dintr-o singură imagine.

Performanță: expresii faciale naturale, dar sincronizare labială inferioară Wav2Lip.

Tehnologii: PyTorch, DeepFace.

2.11 Resurse și Link-uri Oficiale ale Modelelor

Tabelul de mai jos centralizează sursele oficiale pentru toate modelele și tehnologiile investigate în cadrul proiectului, atât cele utilizate în producție, cât și cele testate experimental:

Model / Tehnologie	Link Repository / Sursă
Meta-Llama-3-8B-Instruct	https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct
Mistral 7B (Baseline)	https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3
Wav2Lip (Generare Video)	https://github.com/Rudrabha/Wav2Lip
SadTalker (Testat)	https://github.com/OpenTalker/SadTalker
Piper TTS (Audio)	https://github.com/rhasspy/piper
Whisper (STT)	https://github.com/openai/whisper
Sentence-Transformers	https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
MS-MARCO Reranker	https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2
FAISS Vector DB	https://github.com/facebookresearch/faiss

Tabela 2.1: Referințe oficiale pentru modelele AI utilizate și analizate.

Capitolul 3

Evaluarea sistemului RAG utilizat în Face2Learn

3.1 Descrierea și explorarea datelor (EDA)

Pentru evaluarea componentei de **întrebare-răspuns**, a fost creat manual un set de date format din **50 de perechi întrebare-răspuns**, extrase din manualul `manual2022.pdf`.

Structura fișierului de date (`evaluare.json`):

```
[
  {
    "intrebare": "Ce reprezintă un segment orientat?",
    "raspuns_asteptat": "un segment ... unde s-a precizat originea si extremitatea"
  },
  ...
]
```

Explorare sumară:

- Număr total de exemple: 100;
- Lungime medie întrebare: 9 cuvinte;
- Lungime medie răspuns: 12 cuvinte;
- Domeniu: concepte geometrice (vectori, segmente, egalitate etc.);
- Tip date: text scurt, conceptual – ideal pentru evaluarea unui sistem RAG.

3.2 Descrierea algoritmului inteligent

Algoritm ales: *Retrieval-Augmented Generation (RAG)*.

Motivație: Arhitectura RAG combină avantajele modelelor de căutare contextuală (retrieval) cu cele de generare, eliminând necesitatea antrenării de la zero și reducând halucinațiile.

Componente principale:

- **Retrieval:** Model de embedding `thenlper/gte-small` și indexare cu `faiss-cpu`.
- **Generation:** Modelul `mistral-7b-instruct-v0.3.Q4_K_M.gguf`, rulat local prin LM Studio.

3.3 Metodologia experimentală

Hiperparametri utilizați (Baseline):

Parametru	Valoare	Descriere
chunk_size	256	Lungimea unui fragment de text la indexare
chunk_overlap	25	Suprapunerea dintre fragmente consecutive
k	4	Numărul de pasaje returnate de retriever
temperature	0.3	Controlul creativității LLM-ului

Metrici de evaluare:

- **ROUGE-L (F1):** măsoară suprapunerea exactă între răspunsul generat și cel așteptat.
- **Similaritate Semantică (Cosine Similarity):** În versiunea finală, am înlocuit distanța Euclidiană (L2) cu **Cosine Similarity** pentru indexul FAISS și evaluare.
 - *Justificare:* Distanța Euclidiană măsoară doar distanța dintre puncte în spațiu, fiind sensibilă la magnitudinea vectorilor. În schimb, **Cosine Similarity** măsoară unghiul dintre vectori, ceea ce reflectă mult mai fidel similaritatea semantică și orientarea contextuală a conceptelor, indiferent de lungimea textului.

3.4 Rezultate obținute (Etapa Inițială - Baseline)

Metrică	Valoare medie
ROUGE-L (F1)	20.61%
Similaritate Semantică	90.24%

Concluzie parțială: Sistemul are o performanță semantică excelentă, dar necesită îmbunătățiri la nivel de precizie lexicală și coerență a instrucțiunilor.

Capitolul 4

Îmbunătățirea și Optimizarea Algoritmului Inteligent

În această etapă a proiectului, ne-am concentrat pe îmbunătățirea calității procesului de rezolvare și a complexității spațiale, implementând tehnici avansate de Machine Learning.

4.1 Optimizarea Sistemului RAG prin Two-Stage Retrieval

Pentru a reduce fenomenul de *halucinație*, am implementat o arhitectură de căutare în două etape:

4.1.1 Etapa 1: Candidate Generation (Bi-Encoder)

- **Tehnică:** Căutare Vectorială (*Semantic Search*) folosind FAISS și metrica Cosine Similarity.
- **Model:** `thenlper/gte-small`.
- **Scop:** Maximizarea **Recall-ului** (extragerea a $k = 20$ candidați).

4.1.2 Etapa 2: Reranking (Cross-Encoder)

- **Tehnică:** Reordonare bazată pe relevanță contextuală.
- **Model:** `cross-encoder/ms-marco-MiniLM-L-6-v2`.
- **Proces:** Analizează perechea (Întrebare, Document) simultan, calculând un scor de relevanță precis. Se selectează top 5 documente finale.

4.2 Arhitectura de Căutare Hibridă (Ensembling)

Am implementat o strategie de **Hybrid Search** combinând:

1. **Căutare Semantică (Dense)**: Pentru înțelegerea contextului și a sinonimelor.
2. **Căutare Lexicală (Sparse - BM25)**: Pentru găsirea termenilor exacti (ex: teoreme specifice).

Fuziunea rezultatelor se realizează prin algoritmul **Reciprocal Rank Fusion (RRF)**.

4.3 Optimizarea Modelului Generativ (Integrare QLoRA)

Pentru a adapta modelul la domeniul educațional românesc, am utilizat tehnica **QLoRA** (Quantized Low-Rank Adaptation).

- **LoRA Adapters**: Antrenarea doar a matricelor de rang mic injectate în straturile de atenție ($< 1\%$ parametri).
- **Quantization (NF4)**: Modelul a fost încărcat în precizie de 4-biți, reducând necesarul de memorie VRAM la $\approx 6\text{GB}$.
- **Contribuție**: Integrarea acestor adaptoare permite modelului să adopte o "persona" de profesor, oferind explicații structurate didactic.

4.4 Augmentarea Datelor și a Interogărilor

4.4.1 Query Expansion

Un apel LLM intern generează 3 variante semantice ale întrebării utilizatorului. Sistemul execută căutări paralele pentru toate variantele, îmbunătățind semnificativ acoperirea (Recall).

4.5 Studiu Experimental al Hiperparametrilor

4.5.1 Schimbarea Modelului de Embedding

Trecerea la `paraphrase-multilingual-MiniLM-L12-v2` a îmbunătățit înțelegerea nuanțelor limbii române față de `gte-small`.

4.5.2 Analiza Temperaturii (T)

În urma experimentelor pe setul de validare, am calibrat parametrii pentru a maximiza echilibrul între creativitate și precizie:

- **Temperatura (T):** Valoarea optimă identificată este $T = 0.1$. La $T = 0$, răspunsul devine prea rigid și robotic, în timp ce la $T > 0.2$, scorul ROUGE scade drastic (de la 19.23% la sub 10%), indicând apariția halucinațiilor și devierea de la textul sursă.

4.5.3 Analiza Chunk Size și Top-K

- **Chunk Size:** Dimensiunea de 256 tokeni a oferit cel mai bun echilibru. La 128 tokeni contextul este prea fragmentat pentru a cuprinde definiții complexe, iar la 512 tokeni se introduce zgomot irelevant care diluează atenția modelului.
- **Top-K:** Valoarea $k = 4$ (documente extrase) a fost optimă. O creștere la $k = 6$ a crescut latența sistemului fără a îmbunătăți semnificativ scorul semantic (creștere nesemnificativă de la 91.18% la 91.35%).

Capitolul 5

Arhitectura Sistemului și Detalii de Implementare Software

Pentru a asigura scalabilitatea și ușurința în utilizare, am refactorizat complet codul sursă, trecând de la o execuție script-based la o arhitectură Client-Server modernă și optimizată.

5.1 Arhitectura REST API cu FastAPI

Nucleul aplicației a fost migrat către un server web bazat pe **FastAPI**. Această modificare permite decuplarea logicii de inteligență artificială (Backend) de interfața cu utilizatorul (Frontend/UI).

5.1.1 Definirea Endpoint-urilor

Serverul expune următoarele rute (endpoints):

- **POST /chat**: Endpoint-ul principal care primește întrebarea și returnează răspunsul text și link-ul video.

```
Request: { "message": "Ce este teorema lui Pitagora?" }
Response: {
  "text": "Teorema lui Pitagora afirmă că...",
  "video_url": "http://localhost:8000/videos/response_202405.mp4",
  "sources": ["Manual Pag 45"]
}
```

- **GET /videos/{filename}**: Servește static fișierele video generate.

5.2 Dezvoltarea Interfeței Grafice (Frontend)

Pentru a oferi o experiență de utilizare accesibilă și intuitivă, am dezvoltat o interfață grafică (UI) modernă, utilizând tehnologii web de ultimă generație.

5.2.1 Tehnologii Utilizate

- **React & TypeScript:** Framework-ul principal pentru construirea componentelor UI, oferind modularitate și siguranță a tipurilor de date.
- **Vite:** Utilizat ca build tool pentru performanța sa superioară și timpul redus de încărcare.
- **CSS Modules:** Pentru stilizarea componentelor, implementând un design responsive și estetic.

5.2.2 Design și User Experience (UX)

Interfața a fost construită urmând principiile **Glassmorphism**, caracterizat prin:

- Panouri semi-transparente cu efect de blur (`backdrop-filter: blur(10px)`), oferind un aspect modern și curat.
- Paletă de culori "Dark Mode" (`radial-gradient`) pentru a reduce oboseala ochilor și a evidenția conținutul video.
- Animații fluide pentru apariția mesajelor și a elementelor media.

5.2.3 Funcționalități Cheie Implementate

1. **Efectul "Typewriter":** Componenta `AiMessage` simulează scrierea progresivă a răspunsului AI (caracter cu caracter), imitând un flux conversațional natural și menținând atenția utilizatorului.
2. **Afișare Condiționată (Conditional Rendering):** Videoclipul cu avatarul și lista de surse bibliografice sunt afișate dinamic, doar după ce textul explicativ a fost generat complet, pentru a nu supraîncărca interfața.
3. **Feedback Vizual:** Indicatori de încărcare ("typing dots") informează utilizatorul despre starea procesării pe server.
4. **Comunicare Asincronă:** Frontend-ul comunică cu Backend-ul FastAPI prin cereri asincrone, gestionând erorile de conexiune fără a bloca interfața.

5.3 Optimizări de Infrastructură și DevOps

5.3.1 Unificarea Mediului de Dezvoltare (Single Venv)

Inițial, proiectul necesita două medii virtuale separate. Am reușit unificarea dependențelor (RAG, Llama, Wav2Lip) într-un singur mediu virtual (`venv`), simplificând drastic procesul de instalare și deployment.

5.3.2 Compatibilitate Multi-Platformă (CUDA & Metal)

Codul a fost optimizat pentru a rula pe arhitecturi hardware diverse:

- **NVIDIA (Windows/Linux):** Utilizează CUDA pentru accelerare GPU.
- **Apple Silicon (MacOS):** Am implementat suport pentru **Metal Performance Shaders (MPS)**, permițând rularea bazei de date FAISS și a generării video pe cipurile M1/M2/M3.

5.3.3 Gestionarea Fișierelor Multimedia

- **Timestamping:** Fiecare generare primește un nume unic bazat pe data și ora execuției (ex: `response_TIMESTAMP.mp4`), permițând istoric și acces concurent.
- **Separare Audio/Video:** Sistemul generează fișiere separate pentru debug, iar sincronizarea finală se face într-un container MP4.
- **High-Res Lip Sync:** Am ajustat parametrii Wav2Lip și am implementat un algoritm de super-resolution (post-procesare) pentru a îmbunătăți claritatea zonei bucale, eliminând blur-ul specific modelelor GAN vechi.

5.4 Refactorizarea Codului (Code Cleaning)

Am realizat o restructurare completă a sistemului de fișiere:

- Încărcarea resurselor (modele, index FAISS) se face **o singură dată** la startup-ul serverului ('@app.on_event("startup")'), nu la fiecare cerere, reducând latența inițială de la 15s la <1s per request.
- Codul a fost modularizat în funcții distincte pentru Retrieval, Generation și Animation.

Concluzii

Proiectul „Face2Learn” oferă o abordare inovatoare de integrare a AI multimodal în educație. Prin combinarea LLM-urilor optimizate (Llama 3 cu QLoRA), a tehnicilor avansate de regăsire (Hybrid RAG + Reranking bazat pe Cosine Similarity) și a sintezei video (Wav2Lip High-Res), am creat un sistem capabil să ofere explicații didactice clare, naturale și accesibile.

Arhitectura finală Full-Stack, compusă dintr-un backend robust (FastAPI, CUDA/Metal) și un frontend modern (React, Vite), asigură performanță, scalabilitate și o experiență de utilizare superioară, demonstrând potențialul real al asistenților virtuali în modernizarea procesului de învățare.